Modelling Bacterial Genome Evolution and Activity Through Graph-Theoretic Flow Simulations in Metagenomics

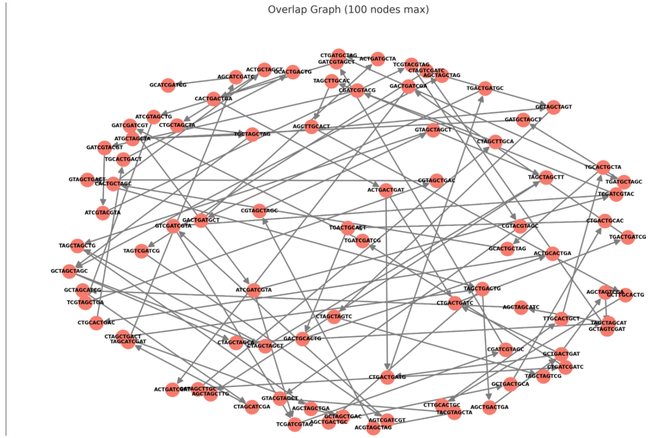
Final Review

Intelligence of Biological Systems & Design and Analysis of Algorithms

Hemesh Yeturu (CB.SC.U4AIE23166)
Joel John (CB.SC.U4AIE23131)
Abhishek S (CB.SC.U4AIE23107)
Adarsh P (CB.SC.U4AIE23109)

IEEE conference purpose

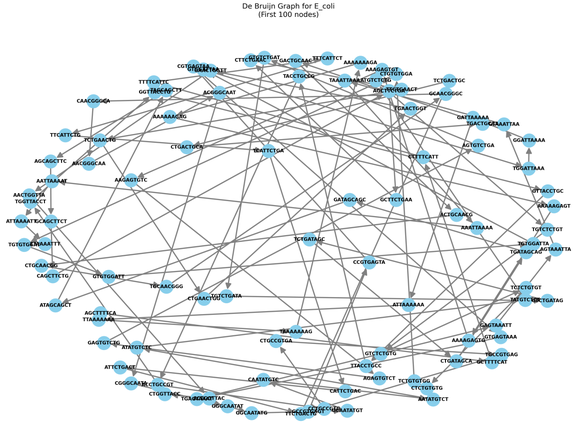
1. Construction of Bacterial Genome Graphs

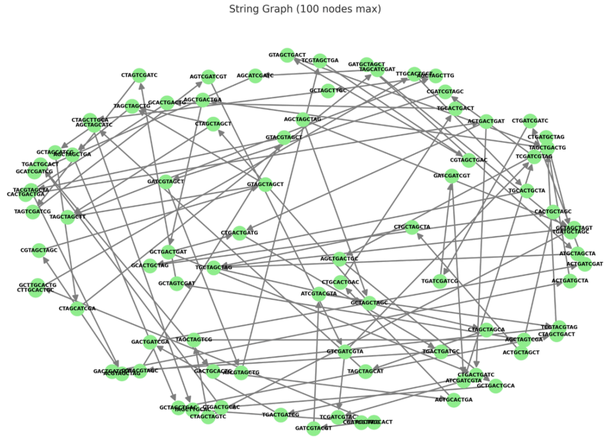


📊 Graph Properties Comparison

- **Edge Count (Fig. 1)**:
  - Overlap Graph: Highest number of edges (e.g., 106 for *B. subtilis*)
  - De Bruijn: Moderate but significant edge density
  - String Graph: Fewest edges → most compact structure
- **Node Count (Fig. 2)**:
  - De Bruijn Graph: Highest node count (unique k

    k-mers → more granular)
  - Overlap & String Graphs: Fewer nodes due to merging
- **Computation Time (Fig. 3)**:
  - De Bruijn Graphs: Fastest construction (due to fixed k-mer parsing)
  - Overlap Graphs: Slowest (due to all-pairs suffix-prefix comparisons)
  - String Graphs: Intermediate (uses reduction for efficiency)





1. Construction of Bacterial Genome Graphs

🧩 **De Bruijn Graph (Fig. 4):**

- Nodes = unique k
  k-mers, Edges = suffix-prefix matches
- Dense connectivity, hard to interpret biologically
- **Strength**: Excellent computational performance
- **Limitation**: Over-fragmentation → reduced clarity

🔗 **Overlap Graph (Fig. 5):**

- Nodes = full reads, Edges = significant overlaps
- Read-based labeling is biologically meaningful
- **Strength**: High interpretability per node
- **Limitation**: Visual clutter due to too many connections

🪄 **String Graph (Fig. 6):**

- Transitively reduced version of overlap graph
- Removes redundant edges, maintains true overlaps
- **Strength**: **Balanced readability & structural accuracy**
- **Limitation**: Slightly higher preprocessing

💡 **Conclusion:**

- **De Bruijn** → Assembly speed & granularity
- **Overlap** → Deep overlap information
- **String Graph** → Optimal clarity for **visual analysis & debugging**

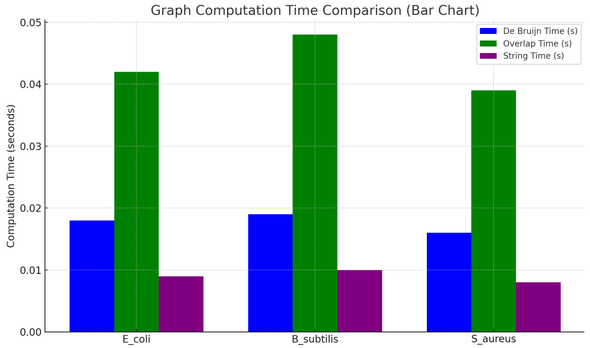


Fig 4

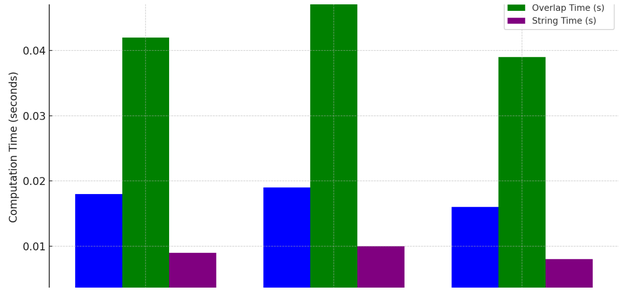


Fig 5

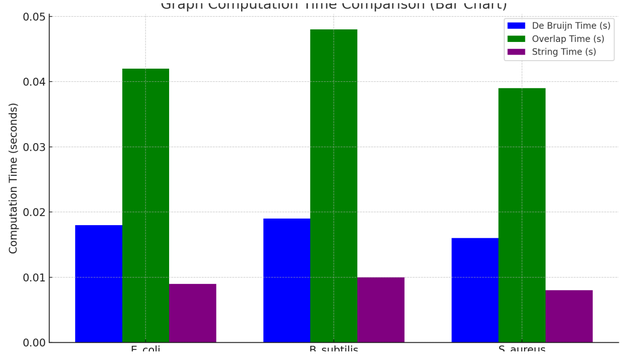
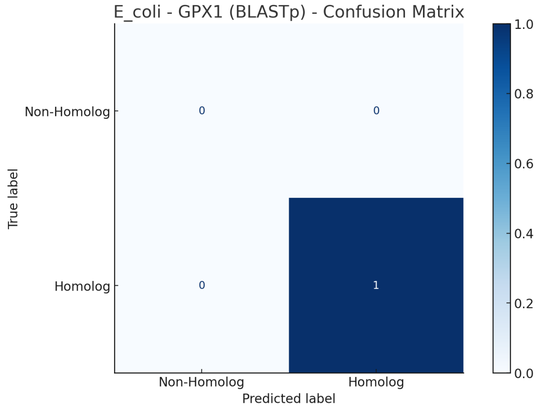
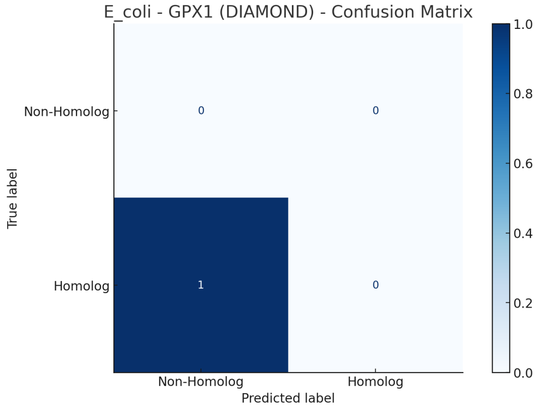
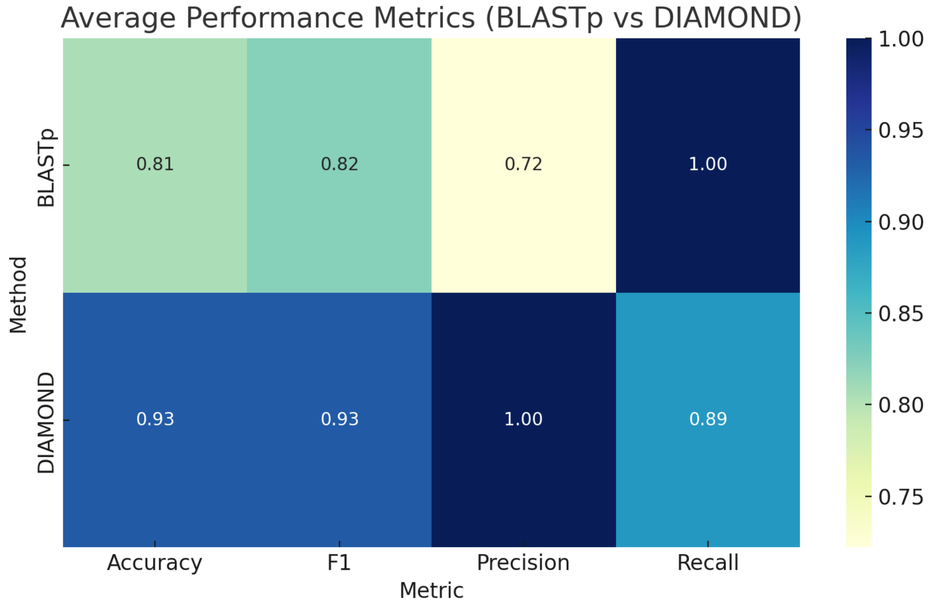


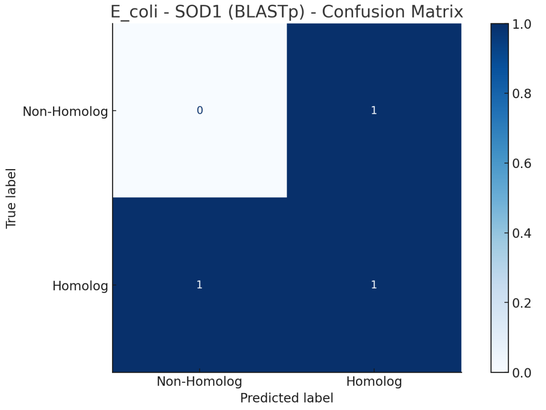
Fig 6

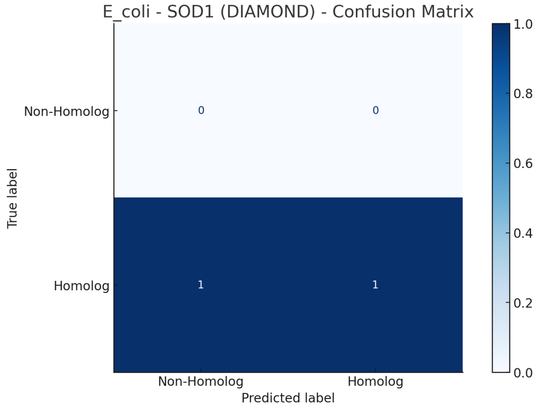
2. ROS Gene Analysis

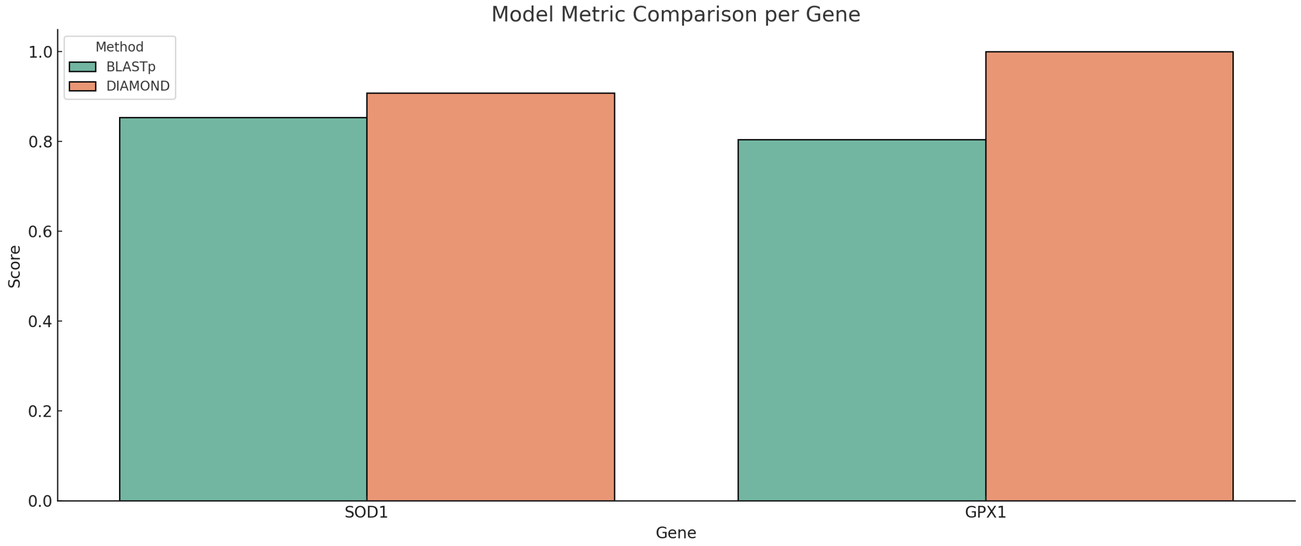












3. Maximum Flow Benchmarking on Genome Graphs

⚙️ **Objective:**

- Benchmark 4 classical **maximum flow algorithms** on real bacterial genome interaction networks:
  - **Edmonds-Karp**
  - **Dinic's Algorithm**
  - **Push-Relabel**
  - **Capacity Scaling**

📊 **Performance Overview (Fig. – Runtime Chart & Table):**

- All algorithms yielded **identical maximum flow**: **2**
- Runtime varied significantly across algorithms

🥇 **Fastest: Capacity Scaling**

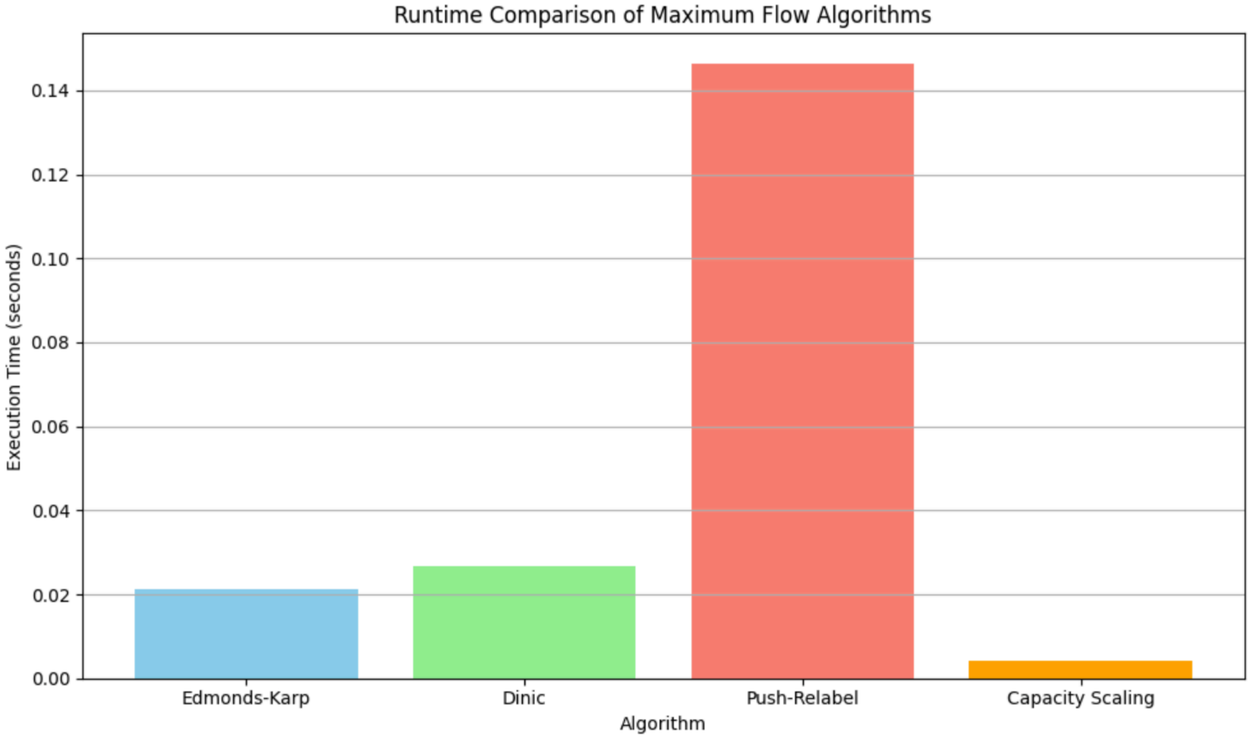
- **Time:** ~0.0022 s
- **Strength:** Optimized for **large-capacity edges**
- **Best suited** for **sparse genome graphs** with clear capacity gaps

⚖️ **Middle Performers:**

- **Edmonds-Karp:** ~0.0094 s
  O(VE2), but performs well due to short augmenting paths
- **Dinic's Algorithm:** ~0.0096 s
  - Uses layered BFS → efficient for moderate graph sizes

🐢 **Slowest: Push-Relabel**

- **Time:** ~0.0181 s
- Relabeling overhead not ideal for **sparse biological graphs**
- **Designed for dense networks** → less effective here



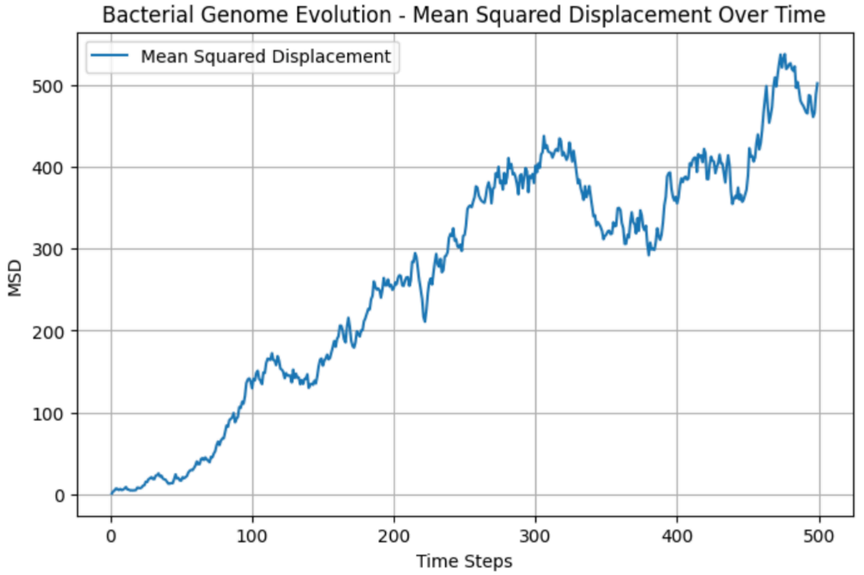
Random Walk Path

🎯 **Objective:**

- Analyze **bacterial genome mobility** and **evolution over time**
- Two core metrics used:
  - **Mean Squared Displacement (MSD)** – for temporal evolution
  - **Spatial Visit Frequency Heatmap** – for spatial colonization dynamics

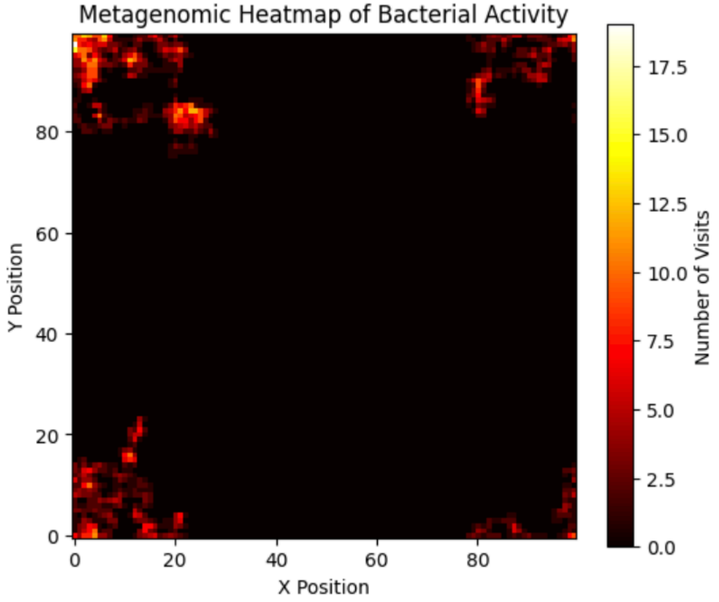
🕒 **Mean Squared Displacement (MSD) – Temporal Evolution**

- **Figure**: MSD over 500 simulation steps (Fig. \ref{fig:msd-evolution})
- **Interpretation:**
  - Upward MSD trend → **continuous genome drift** due to evolution
  - Plateau phases → **genetic stability** or environmental constraint
  - Final MSD > 500 → **significant divergence** from original genome
    - Implies mutations, gene gains/losses, or horizontal gene transfer



🌍 **Heatmap of Spatial Genome Activity**

- **Figure**: Spatial heatmap of genome visit frequency (Fig. \ref{fig:heatmap-activity})
- **Grid Size**: 100x100 metagenomic landscape
- **Key Patterns:**
  - **Red/Yellow Zones**: High-frequency genome presence
    - Indicates microbial **hotspots**, niche colonization, biofilm zones
  - **Dark Zones**: Genome-absent regions
    - Reflect **spatial exclusion** or environmental unsuitability



💡 **Insights and Conclusions:**

- Bacterial evolution is **non-uniform and stochastic**
- **MSD** shows **temporal genetic drift**
- **Heatmap** reveals **spatial colonization preferences**
- Supports models of:
  - Genome flow across ecological niches
  - Potential **mutation reservoirs**
  - Selective evolution and niche stabilization

Quantitative Comparative Analysis

⚙️ **Algorithm Insights**

- **Edmonds-Karp**
  - Simple BFS-based approach
  - Struggled on dense Overlap graphs due to slower augmenting paths
- **Dinic's Algorithm**
  - Performed best on String Graphs
  - Balanced performance thanks to moderate density and reduced transitivity
- **Push-Relabel (Generic)**
  - Worked well on Overlap graphs with uniform high degrees
  - Slower in sparse topologies due to unnecessary relabeling
- **Capacity Scaling**
  - Enhanced version of Push-Relabel for integer capacities
  - Highly efficient on graphs with normalized edge capacities (especially String Graphs)
- **Relabel-to-Front**
  - Push-Relabel variant with aggressive relabeling strategy
  - Fast convergence in graphs with many small-capacity paths
- **FIFO Push-Relabel**
  - Processes active nodes in first-in-first-out order
  - Showed consistent, stable runtimes across De Bruijn and String Graphs
- **Ford-Fulkerson**
  - Theoretically sound but impractical for real datasets
  - Performance suffered due to possible infinite loops on irrational capacities

🏆 **Conclusions**

- **Best performers**: Dinic’s and FIFO Push-Relabel on biologically relevant graphs
- **Ideal for**: Evolution modeling and microbial flow simulations
- **Avoid**: Ford-Fulkerson and Edmonds-Karp on dense or irregular topologies

CONCLUSIONS & FUTURE SCOPE

- This study presents a comprehensive comparative analysis of three prominent genome graph models—De Bruijn, Overlap, and String graphs—applied to microbial genomes of E. coli, B. subtilis, and S. aureus. Our findings highlight the distinct trade-offs in structural complexity, interpretability, and graph compactness across models, with String graphs offering a balanced representation that preserves biological clarity while reducing redundancy. By benchmarking BLASTp and DIAMOND for the detection of redox-related genes such as GPX1,SOD1, and CAT, we demonstrate that functional gene detection is influenced not only by tool selection but also by underlying genome structure. The inclusion of classical and advanced maximum flow algorithms on genome-derived graphs further bridges the gap between computational theory and biological application, showing how signal propagation and gene connectivity can be effectively modeled using flow dynamics.

- The insights gained from this work pave the way for more biologically informed graph algorithms and models. In the future, this framework can be extended to metagenomic or viral datasets, where structural diversity is even more pronounced. Moreover, integrating transcriptomic or epigenetic data into these graph structures could enable dynamic modeling of regulatory changes. The evolution simulation module can also be enhanced using probabilistic flows or reinforcement learning to better capture adaptive genomic shifts. Overall, this work provides a unified and extensible platform for exploring the intersection of genome graph theory, functional genomics, and algorithmic network analysis in microbial systems.